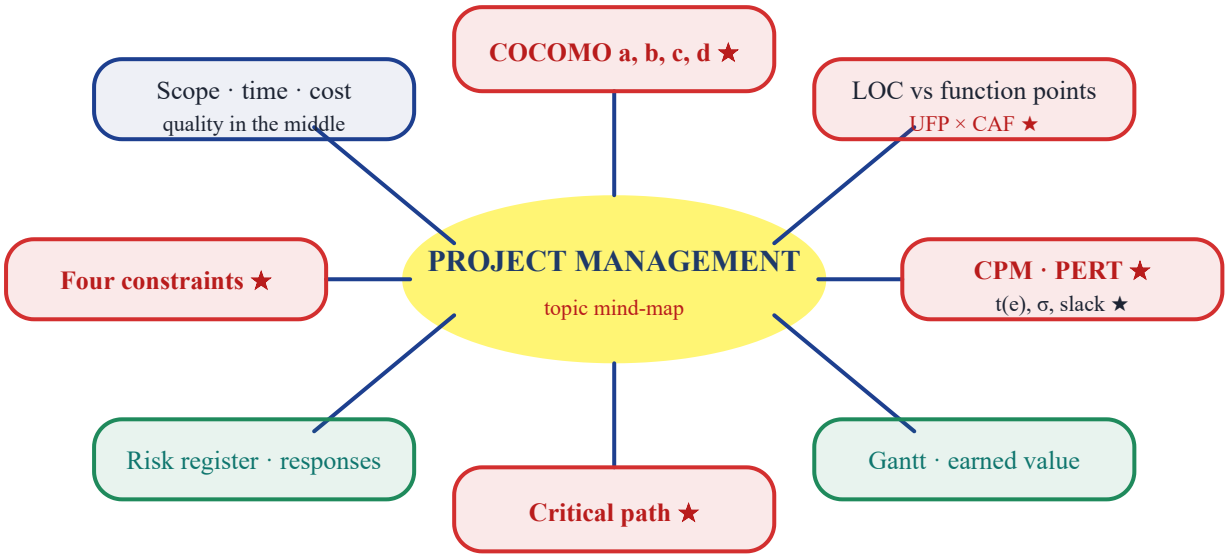
Project Management & Scheduling

Metrics · COCOMO · Function Points · Risk · CPM/PERT · Gantt: exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = method / applied

Syllabus coverage

"**Project Management Concepts**, **Software process and project Metrics**, **Software Project Planning**, **Risk Analysis and management**, **Project Scheduling and Tracking**." This notebook owns those five lines. SDLC phases and paradigms → SE Concepts; analysis and UML → Analysis & Design; testing and quality metrics → Testing & Quality.

High-yield list

① scope · time · cost · quality
② COCOMO $E = a \cdot (KLOC)^b$, $T = c \cdot E^d$
③ FP: EI EO EQ ILF EIF
④ $t_e = (o+4m+p)/6$
⑤ slack = LS − ES = LF − EF
⑥ exposure = P × impact

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

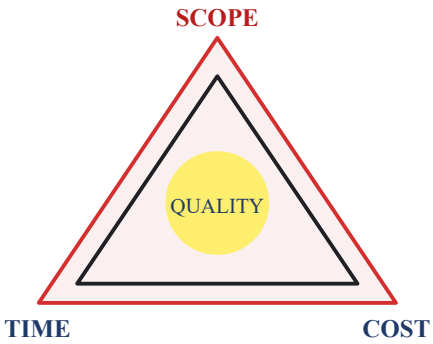
- ✓ What project management is; the four constraints ..... p. 2
- ✓ The project activities ..... p. 3
- ✓ Metrics: LOC vs FP, productivity ..... p. 4
- ✓ Function Point Analysis, worked ..... p. 5
- ✓ Estimation & COCOMO, worked ..... p. 6
- ✓ Risk management ..... p. 7
- ✓ CPM: network, slack, critical path ..... p. 8
- ✓ PERT, Gantt, monitoring & control ..... p. 9
- ✓ Night-before sheet ..... p. 10
- ✓ Practice MCQs + written ..... p. 11

1 Software Project Management & the Four Constraints

Definition: **Software Project Management (SPM)** = the application of management knowledge, skills, tools and techniques to software projects so that they are delivered **on time, within budget, and to the required quality**. A **project** is a **temporary** endeavour producing a **unique** result — unlike "operations", which repeat.

Constraint	Question it answers	Driven by
Scope	what work is **in**, what is out	requirements, WBS, SRS
Time	how long, by which **milestones**	estimates, CPM / PERT, Gantt
Cost	how much money and effort	budget, COCOMO, rates
Quality	how good — the **result** of the other three	SQA, reviews, testing, CM

Most texts state the **triple constraint** (scope–time–cost, a triangle with the project inside) and then note that **quality sits in the middle** and is the first casualty. **Fix three and the fourth is decided for you.**



tighten one leg → another must give

the triple-constraint triangle

Exam trap: The trade-off is a **diagnosis**, not an excuse. Shrink **scope** before you shrink quality; buy **time** before you buy quality.

2 Who Does What — and the Baseline

- **Project Manager** — plans, schedules, budgets, reports; owns delivery
- **Sponsor / Owner** — funds it, holds the business case
- **Product Owner** — owns the backlog and the priority of change
- **PMO** — Project Management Office: standards, templates, audits
- **CCB** — Change Control Board: the only body that may move the baseline

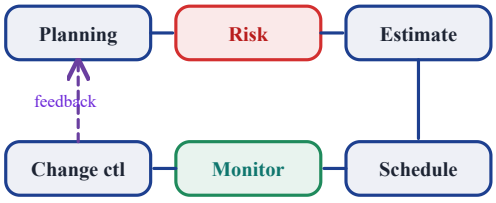
Scope & change control: **Scope creep** = uncontrolled growth of the work with no change to time or cost. The cure is **configuration management (CM)**: versioning, baselines, change requests, configuration audits, status accounting.

Memory trick: The **baseline** is the approved scope–schedule–cost. Nothing is a "change" until it is **approved against a baseline**. The PM changes the plan; the **CCB** approves it; the **PO** re-prioritises the backlog.

3 The Project Activities

Activity chain: Every project manager runs the same loop: **planning → risk analysis → estimation → scheduling → monitoring & control**, with **configuration / change management** wrapping all of it. These are **not** SDLC phases (that is the SE Concepts lesson); they are the manager's activities across the whole life of the product.

#	Activity	What it produces
1	**Planning**	objectives, scope statement, WBS, staffing, communication plan
2	**Risk analysis**	risk register: identify → assess → prioritise → respond
3	**Estimation**	size (KLOC / FP), effort, cost — LOC, FP, COCOMO, analogy
4	**Scheduling**	network, critical path, Gantt chart, milestones, baseline
5	**Monitoring & control**	progress vs plan, variance, corrective action
6	**Configuration / change mgmt**	baselines, version control, change requests, CCB



the manager's loop — never a straight line

Technique	How it works
Analogy (top-down)	extrapolate from a finished project; **least accurate**
Expert judgement	experts estimate from experience — fast, optimistic
Delphi	same, but **anonymous**, seeing only the spread

Technique	How it works
Bottom-up (WBS)	estimate every work package, add overhead
FP + COCOMO	language-free size → effort and schedule
Planning Poker · **price-to-win**	story points by card mode; tender price

Memory trick: WBS: **W**ork **B**reakdown **S**tructure: the hierarchical decomposition of the project into **work packages** small enough to estimate, assign and measure (≈ 80 person-hours). Every leaf must be **100 % assigned** — no "miscellaneous" leaf.

Exam trap: **Effort ≠ duration.** Effort is person-time (person-months), duration is calendar time. Ten people for one month = 10 person-months of effort but 1 month of schedule. Adding people to a late project usually makes it **later** (Brooks's law).

4 Process & Project Metrics

Definition: A **metric** is a **quantitative measure** of size, process effectiveness, product quality or project performance. A **baseline** is the agreed value it is compared against.

Family	Question it answers	Typical measures
Product (size)	how big is the software?	**LOC**, function points, cyclomatic complexity, Halstead
Process	how well are we developing?	effort, productivity, defect density, estimation accuracy
Project	how is the project doing?	schedule variance, cost variance, earned value (SPI, CPI)
Quality	how good is the delivered product?	defects per KLOC / per FP, MTBF, MTTR, availability

Size: LOC vs function points, and the derived measures

defect & reliability metrics in depth → Testing & Quality lesson

	LOC	Function points
Counts	lines of source code	user-visible functions
Language	**dependent**	**independent**
Available	only **after** coding	from the **spec**
Is it size?	no — a shortcut	yes, FPs **are** size
Weakness	rewards verbose style	judgement-heavy

Derived measures — formulas:

Productivity = **LOC ÷ person-months** · Defect density = **defects ÷ KLOC** · Defect removal efficiency = **found before delivery ÷ total**

Estimation error = **(actual − estimated) ÷ estimated × 100 %** · MTBF = **1 ÷ failure rate** · Availability A = **MTBF ÷ (MTBF + MTTR)**

Worked example ✍️: 2000 LOC with 10 defects ⇒ KLOC = 2, defect density = 10 ÷ 2 = **5 per KLOC**. 12,000 LOC in 15 person-months ⇒ productivity = **800 LOC/person-month**. 20 defects over 400 FP ⇒ **0.05 defects/FP**, language-free.

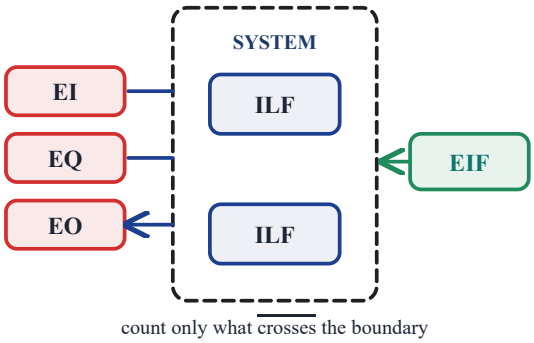
Why LOC productivity is contested ★: (i) the same function can take **3× more lines** in COBOL than in Java; (ii) verbose code **raises** LOC and so **raises** the "productivity" number; (iii) it is unavailable exactly when the estimate is needed. Use **FP, feature or use-case points**.

Exam trap: **LOC is not a size measure** in the ISO/IFB sense; **FPs are.** And "defects per KLOC" over **test code** is meaningless — test modules sit inside KLOC but their defects are not product defects.

5 Function Point Analysis

Definition: **Function Point Analysis (FPA)** — A. J. Albrecht, late 1970s — measures a system by the **functionality a user can see**, not by the code. It counts **five** element types at the **system boundary**, weights them, sums to a **UFP**, adjusts for complexity, and the result is the function point count.

Type	Count	What it is
EI external input	× **3**	unique data **into** the system from outside
EO external output	× **4**	unique data **out** that is **derived** (report, calculated screen)
EQ external inquiry	× **2**	a **read-only** retrieval, no derivation — a search or lookup
ILF internal logical file	× **7**	a **user-identifiable** group of data **inside** the boundary
EIF external interface file	× **5**	user-identifiable data **outside**, maintained elsewhere



the five element types around the boundary

Two equations:

UFP = Σ (count of each type × its weight) · **FP** = **UFP** × **CAF**

CAF: ≤ 50 % improvement → 0 · 51–75 % → **0.40** · 76–100 % → **0.60** · 101–150 % → **0.85** · > 150 % → **1.00**

Worked example ✍️: Payroll system: EI = 20, EO = 8, EQ = 12, ILF = 6, EIF = 3.

1. UFP = 20×**3** + 8×**4** + 12×**2** + 6×**7** + 3×**5** = 60 + 32 + 24 + 42 + 15 = **173**
2. Complexity improvement potential **80 %** ⇒ CAF = **0.60**
3. FP = 173 × 0.60 = 103.8 ≈ **104 function points**; roughly 104 × 40 LOC ≈ **4.2 KLOC** for COCOMO

Memory trick: Weight order, high to low: **ILF 7 > EIF 5 > EO 4 > EI 3 > EQ 2** — "the **I**nternal **L**ogical **F**ile is the heavyweight, the inquiry the featherweight". A field that arrives without changing behaviour is **not** an EI; a report that merely repeats stored data is an **EQ**, not an EO.

Exam trap: There are **five** element types, not six — "external inquiries" is the third, not an extra one. CAF **multiplies**; it is never added. FP counts the user's view, so it must be re-counted whenever the boundary moves.

6 Estimation & COCOMO

COCOMO (Boehm, 1981): **Constructive Cost Model**: a parametric model that turns a size measure into **effort** and then into **schedule**. Effort is always person-months; schedule is always months.

The two equations:

Effort (E) = a × $(KLOC)^b$ × ∏EM

∏EM = product of the effort multipliers

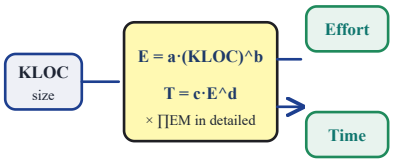
Schedule (T_{DEV}) = c × $(E)^d$

b > 1 ⇒ effort grows **faster** than size

Project mode	Basic	Full	Det.	b	c	d
Organic	**2.5**	3.2	3.2	1.05	2.5	0.38
Semi-detached	3.0	3.0	3.0	1.12	2.5	0.35
Embedded	**3.6**	4.3	4.3	1.20	2.5	0.32

Organic — small, familiar, few rigid rules · **Semi-detached** — medium, mixed team, some rules · **Embedded** — tight constraints, hardware and regulations.

Basic = size only. **Full / intermediate** = size × mode. **Detailed** adds ∏EM: product, platform, personnel and project multipliers (≈ 0.75–1.40, worst case 0.65), so the effective **a** rises to about **3.2** for organic. COCOMO II (1997) drops the modes for size factors including **schedule stretch**.



size in, effort and time out

Worked example — organic, 10 KLOC ✍️:

1. $10^{1.05} \approx 11.220$; E = 2.5 × 11.220 = **28.05 person-months** (a = 2.5, b = 1.05)
2. T_{DEV} = 2.5 × $(28.05)^{0.38}$ = 2.5 × 3.549 = **8.87 months**; average staffing = 28.05 ÷ 8.87 ≈ **3.2 people**

Second example ✍️: Detailed organic, a = 3.2: E = 3.2 × 11.220 = 35.90 ≈ **35.9 person-months**, T_{DEV} = 2.5 × $35.9^{0.38}$ ≈ **9.7 months**. If the question supplies **its own a**, use that a.

Memory trick: Read the constants as a story: a harder project ⇒ bigger **a** (more effort per KLOC) and bigger **b** (effort grows faster). Bigger **d** = more time per unit effort. And **c = 2.5 everywhere**.

Worked example 3 — detailed mode ✍️: Embedded project, 8 KLOC; ∏EM = 0.95 × 0.85 × 1.10 × 1.20 = **1.066**; $8^{1.20} \approx 12.13$. E = 4.3 × 12.13 × 1.066 = 4.3 × 12.93 ≈ **55.6 person-months**; T = 2.5 × $(55.6)^{0.38}$ ≈ **11.5 months**.

Using it in practice: Choose the mode from the **constraints**, not from the wish — only an avionics display is truly **embedded**. Re-baseline whenever the specification changes: that is normal management, not failure.

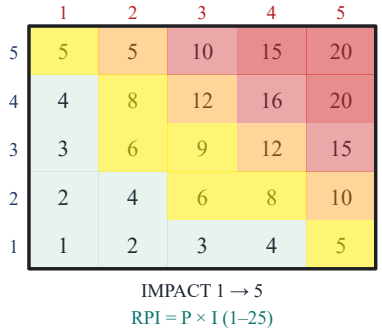
Exam trap: Basic COCOMO has **three** modes — organic, semi-detached, embedded; **"distributed" is not one**. **c = 2.5 in every mode**; only a, b and d change. The schedule equation takes **E** (person-months), never KLOC.

7 Risk Analysis & Management

Definition: A **risk** is an **uncertain event** that, if it occurs, affects a project objective (scope, schedule, cost, quality). It is characterised by **probability × impact**. **Before** it happens it is a risk; **after** it happens it is an issue.

Category	Typical risks
Technical	unfamiliar technology, performance shortfall, security flaw
Organisational	key staff leave, poor training, weak management, legal change
Project mgmt	schedule slip, **scope creep**, cost overrun, staff turnover
External	market shift, vendor price change, currency, natural disaster

Sommerville folds these into three: technical · organisational · project, with external inside organisational.



risk priority matrix — cells show RPI

The five-step loop

1. **Identify** — checklist, brain-storm, Delphi, lessons learned
2. **Assess** — probability (very low … very high) × impact
3. **Prioritise** — sort by RPI / exposure; keep a written **risk register**
4. **Respond** — pick a strategy, name an **owner** and a trigger
5. **Monitor** — review at every status meeting; re-assess and close

The four responses

Response	Meaning
Avoid	remove the threat — change the plan so it cannot happen
Reduce / mitigate	lower probability or impact — prototype, extra testing, training
Transfer	hand the **consequence** over — insurance, warranty, subcontract
Accept	do nothing but hold a **contingency reserve** and a fallback

Worked example ✍️: A production server has P = 0.3 and costs Tk 50,000 to replace if it fails. **Risk exposure** RE = P × C = 0.3 × 50,000 = **Tk 15,000** — that is the amount to hold as contingency. Size the contingency reserve from the **sum** of the exposures of all high-priority risks, not from one big risk.

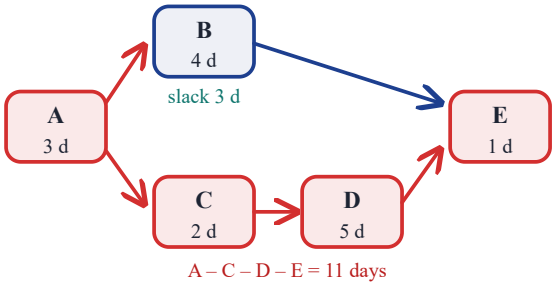
The two reserves: **Management reserve** covers the **known-unknown** risks, inside the baseline. **Contingency reserve** covers the **unknown-unknown** risks, outside it.

Memory trick: The risk **score** (P × I) **ranks**; the risk **exposure** (P × cost) **sizes the reserve**. Never manage by exposure alone.

8 Scheduling I — CPM, Critical Path & Slack

CPM (DuPont + IBM, 1957-58): Draw every activity as a **node** with its **duration written inside** in an **AON network** (activity-on-node), then walk it **forward** for the earliest times and **backward** from the deadline for the latest times. The path of zero slack is the **critical path**, and its length **is** the project duration.

Act	Dur	Pred	ES	EF	LS	LF	Slack
A	3	—	0	3	0	3	**0**
B	4	A	3	7	6	10	**3**
C	2	A	3	5	3	5	**0**
D	5	C	5	10	5	10	**0**
E	1	B, D	10	11	10	11	**0**



red = critical · blue = 3 d float

Forward pass: ES = max(EF of all predecessors); EF = ES + duration. **Backward pass:** LF = min(LS of all successors); LS = LF − duration. Project finish F = max(EF). **Total float = LS − ES = LF − EF.**

Worked example ✍️: Two paths: A–B–E = 3 + 4 + 1 = **8 d**; A–C–D–E = 3 + 2 + 5 + 1 = **11 d** ⇒ the critical path is **A–C–D–E** and the project takes 11 days. E must start by day 10, so B's latest finish is 10 while its earliest finish is 3 + 4 = 7 ⇒ **float of B = 10 − 7 = 3 days**. Delay B by 4 days and the project still finishes on time.

Dependencies between activities

Type	Means
FS finish-to-start	B starts only when A has **finished** — the default
SS start-to-start	B starts only when A has **started**
FF finish-to-finish	B may finish only when A has **finished**
SF start-to-finish	B may finish only when A has **started** — rare

Memory trick: Name it **what happens first → what happens next**. **F** = the first activity is **finished** before the second begins (FS); **S** = it has only **started**.

Exam trap: **Total float** = how long an activity may slip without delaying the project; **free float** = how long it may slip without delaying an **immediate successor**. Free float ≤ total float. The critical path is the **longest** path — "fewest activities" is a red herring.

IT link: A build pipeline has the same shape: `lint → test → package` is a critical chain, a documentation job running in parallel has float, and only shortening the critical chain moves the release date.

9 Scheduling II — PERT, Gantt & Control

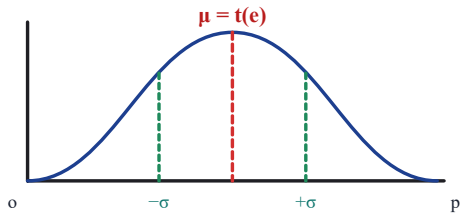
PERT (Malcolm Rose, Polaroid, 1958): The same network as CPM, but every duration is **uncertain**, so it is given as three estimates — optimistic **o**, most likely **m**, pessimistic **p** — and converted to an **expected time** plus a **standard deviation**.

The two PERT equations:

$t_e = (o + 4m + p) \div 6$ (β-type expected value)

$\sigma = (p - o) \div 6$ · variance = σ^2

Act	o	m	p	t_e	σ
A	2	5	11	**5.5**	1.50
B	3	6	9	6.0	1.00
C	4	7	16	**8.0**	2.00
D	1	3	5	**3.0**	0.67



t(e) = (o + 4m + p)/6

σ = (p − o)/6

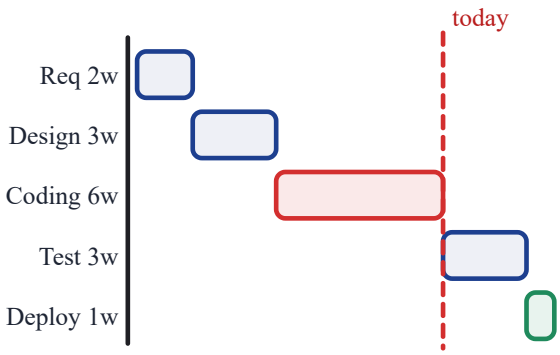
the PERT beta distribution

Network A→B, A→C, B→D, C→D. Path A–B–D = 14.5 d; path A–C–D = 5.5 + 8 + 3 = **16.5 d** ⇒ critical path A–C–D, float of B = **2 d**. Project σ = $\sqrt{(1.5^2 + 2.0^2 + 0.67^2)} = \sqrt{6.69} \approx$ **2.59 days** — the finish is 16.5 ± 2.6 days.

The Gantt chart and monitoring

Gantt chart (Henry Gantt, 1910s) — bars on a time axis, one row per activity: length = duration, position = start. Great for **progress tracking**, but it shows **no** dependencies or float, so it can hide a critical path.

Tool	Best for	Blind to
Gantt	who is doing what, when	dependencies, slack
CPM	deterministic dates, critical path	uncertainty
PERT	probabilistic dates, slippage risk	resource loading, cost



bar = duration · gaps = float

bar position = start, bar length = duration

Earned value (EVM): **PV** planned value · **EV** earned value · **AC** actual cost. SV = EV − PV · CV = EV − AC · SPI = EV ÷ PV · CPI = EV ÷ AC. PV = 100 000, EV = 85 000, AC = 95 000 ⇒ SV = **−15 000**, CV = **−10 000**, CPI = **0.89**.

Memory trick: **Control** is not **inspection**: **monitor** = observe against the baseline, **control** = correct the deviation.

Night-Before-the-Exam Sheet

Fact sheet:

Triple constraint: scope · time · cost, quality inside

Baseline = approved scope–schedule–cost

COCOMO: $E = a \cdot (KLOC)^b \cdot \prod EM$; $T = c \cdot E^d$

Modes: organic · semi-detached · embedded

Organic a = 2.5 / 3.2 / 3.2 · b = 1.05 · d = 0.38

Semi-detached a = 3.0 · b = 1.12 · d = 0.35

Embedded a = 3.6 / 4.3 / 4.3 · b = 1.20 · d = 0.32

c = 2.5 in **every** mode

FP weights: ILF 7 · EIF 5 · EO 4 · EI 3 · EQ 2

FP = UFP × CAF (0.40 / 0.60 / 0.85 / 1.00)

PERT $t_e = (o + 4m + p)/6$ · $\sigma = (p - o)/6$

Float = LS − ES = LF − EF; critical = 0

ES = max(EF preds) · EF = ES + d

LF = min(LS succs) · LS = LF − d

Risk exposure = P × impact

RPI = P(1-5) × I(1-5), max 25

Responses: avoid · reduce · transfer · accept

SV = EV − PV · CV = EV − AC

Top 12 one-liners for MCQs

1. Basic COCOMO has **three** modes, none called distributed

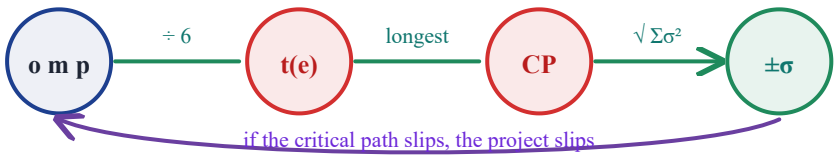
7. $t_e = (o + 4m + p)/6$

8. $\sigma = (p - o)/6$

2. FP counts the **user's** view, not the code
3. Five FP types; ILF carries weight 7
4. CAF multiplies the UFP, never adds
5. LOC is **not** a size measure; FP is
6. Defect density = defects ÷ KLOC
9. Critical path = **longest** path, zero slack
10. Total float = LS − ES = LF − EF
11. Exposure = probability × impact
12. Gantt shows duration, **not** dependencies

Top traps: "Distributed" is not a COCOMO mode · effort is person-months, duration is months · the schedule equation takes E, not KLOC · there are five FP types, not six · an output with no derivation is an EQ · slack ≠ free float · critical path is the longest, not the shortest · a risk **score** is probability × impact, not a probability alone

All tricks together: Fix three constraints, the fourth is decided · a change is a change only against a baseline · WBS leaves must be 100 % assigned · add people to a late project and it gets later · ILF is the heavyweight, EQ the featherweight · read the constants as a story, c = 2.5 always · name the dependency "first happens, then happens" · zero float means critical · o, 4m, p is the PERT weight · transfer the consequence, not the probability



from three guesses to a dated, risky commitment

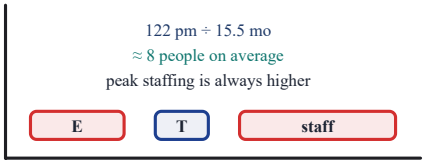
✓ Practice MCQs (exam pattern, 10 marks)

1. Which is NOT a mode in Basic COCOMO?
   (a) Organic (b) Semi-detached
   (c) Embedded (d) Distributed
2. Organic, $E = 2.4 \times (KLOC)^{1.05}$, KLOC = 10. Effort:
   (a) ≈ 24 pm (b) ≈ 27 pm
   (c) ≈ 32 pm (d) ≈ 41 pm
3. P = 0.3, cost if it occurs Tk 50,000. Exposure:
   (a) Tk 15,000 (b) Tk 35,000
   (c) Tk 50,000 (d) Tk 150,000
4. Which size metric is language-independent?
   (a) LOC (b) Function points
   (c) Cyclomatic V(G) (d) Defect density
5. A(3) → B(4), C(2) → D(5) → E(1) after B, D. Float of B:
   (a) 0 days (b) 2 days
   (c) 4 days (d) 3 days
6. PERT: o = 2, m = 5, p = 11. Expected time:
   (a) 5.5 days (b) 6.0 days
   (c) 5.67 days (d) 6.5 days
7. How many FP element types are counted?
   (a) 3 (b) 4
   (c) 5 (d) 6
8. The critical path of a project is:
   (a) the shortest path (b) fewest activities
   (c) longest path, zero float (d) highest cost
9. PERT standard deviation of a task:
   (a) (p − m)/6 (b) (p − o)/6
   (c) (m − o)/6 (d) (p − o)/4
10. Which is NOT a response to a threat?
    (a) Exploit (b) Reduce / mitigate
    (c) Transfer (d) Avoid

Answer key → 1-d · 2-b · 3-a · 4-b · 5-d · 6-a · 7-c · 8-c · 9-b · 10-a

Written question: An organic-mode project of 32 KLOC is to be estimated with COCOMO using a = 3.2, b = 1.05, c = 2.5, d = 0.38. Find the effort, the development time and the average team size. Then, for the network A(3) → B(4) and C(2), C → D(5), E(1) after B and D, find the critical path, the project duration and the total float of B. 10 marks

1. $32^{1.05} \approx$ **38.05**
2. E = 3.2 × 38.05 = **122 person-months** 3
3. T = 2.5 × $(121.8)^{0.38}$ = **15.5 months** 3
4. Staff = 121.8 ÷ 15.5 ≈ **8 people** 1
5. A–B–E = 8 d; A–C–D–E = 11 d ⇒ **critical = A–C–D–E, 11 d** 2
6. E starts by day 10 ⇒ LF(B) = 10, EF(B) = 7 ⇒ **float(B) = 3 d** 1



effort → time → headcount; state the **mode** and the **size** first

Before the exam ✓

- ✓ Can name the four constraints and explain the trade-off
- ✓ Can compute UFP and FP from raw counts
- ✓ Can run COCOMO, effort and schedule, for all three modes
- ✓ Can do a forward and backward pass and find total float

Also practise (written):

① FP for a library system, 80 % complexity potential

② COCOMO detailed mode with ∏EM = 0.85

③ PERT project σ over a 3-activity critical path